

SQL Injection - teoria e spiegazioni

Attacchi SQL Injection (SQLi)

La **SQL Injection (SQLi)** è un grave difetto di architettura web che si verifica quando un'applicazione accetta l'input dell'utente e lo concatena direttamente all'interno di una query di database senza alcuna validazione o sanificazione.

Questo difetto permette a un attaccante di alterare la semantica della query (spesso terminando prematuramente una stringa di testo e accodando un nuovo comando), forzando il database a eseguire istruzioni impreviste. L'obiettivo più comune è l'estrazione massiva di dati, ma a seconda dell'ambiente l'attacco può portare alla modifica/eliminazione di dati, all'esecuzione di comandi del sistema operativo o ad attacchi Denial-of-Service (DoS).

L'Impatto sulla Sicurezza: Violazione della Triade CIA

L'impatto critico della SQLi si analizza attraverso la compromissione della **Triade CIA** (Confidenzialità, Integrità, Disponibilità):

- **Violazione della Confidenzialità:** Un attaccante non autorizzato riesce a leggere dati sensibili (es. credenziali, carte di credito). Questo avviene solitamente tramite tecniche In-Band (usando la clausola **UNION** per accodare risultati estranei o lo schema del database) o Error-Based (forzando il database a restituire errori parlanti tramite funzioni come **ExtractValue()**).
- **Violazione dell'Integrità:** I dati o i processi di sistema vengono alterati. Questo si riflette nell'*Authentication Bypass* (usando una tautologia come **'OR 1=1 #** per ingannare la logica di login) o nella *Data Alteration* (iniettando comandi **UPDATE** per sovrascrivere campi, forzando ad esempio un cambio email per un Account Takeover).
- **Violazione della Disponibilità:** Il servizio viene reso inaccessibile. Qui entra in gioco la tecnica delle *Piggybacked Queries* (o Stacked Queries), dove l'attaccante accoda un comando distruttivo, come un **DROP TABLE**, utilizzando il terminatore di istruzione (;).

Vettori di Attacco (SQLi Attack Avenues)

Gli attaccanti possono sfruttare diversi punti di ingresso per iniettare le loro query:

Vettore	Descrizione
Input dell'utente	Iniezione di comandi fornendo input appositamente manipolati nei form web.
Variabili del Server	Falsificazione dei valori negli header HTTP/di rete per passare dati malevoli bypassando i form.
Second-order Injection	L'attacco sfrutta dati malevoli precedentemente salvati nel sistema. Si innesca solo quando il sistema riutilizza quel dato per costruire una nuova query.
Cookie	Alterazione del contenuto dei cookie. Se il server costruisce una query basata su di essi, la query viene manipolata.
Input Fisico	Applicazione di input tramite canali che esulano dalle tradizionali richieste web.

Categorie di Attacchi

Gli attacchi si classificano in base a come l'attaccante riesce a estrarre i dati dal database.

In-Band SQLi

L'attaccante utilizza lo stesso canale di comunicazione sia per eseguire l'attacco che per raccogliere i risultati. I dati esfiltrati vengono presentati direttamente nella pagina web.

- **Union-Based:** Si utilizza l'operatore **UNION** per combinare i risultati della query originale con i risultati di una query iniettata. Affinché funzioni, la query iniettata deve avere lo stesso numero di colonne e tipi di dato compatibili con quella originale.
- **Error-Based:** Si inviano input volutamente errati (sintatticamente o logicamente), forzando il database a generare un messaggio di errore. Questi errori spesso contengono frammenti di dati o informazioni sulla struttura del DB.
- **Tautologia:** Iniezione di codice nelle istruzioni condizionali (es. **WHERE**) affinché restituiscano sempre un valore **TRUE** (es. **'OR 1=1 --'**).
- **End-of-line comment:** Uso di commenti (**--** o **#**) per annullare il codice legittimo successivo all'iniezione.
- **Piggybacked queries:** Aggiunta di query multiple oltre a quella originale, concatenate tramite il terminatore di istruzione.

Inferential SQLi (Blind)

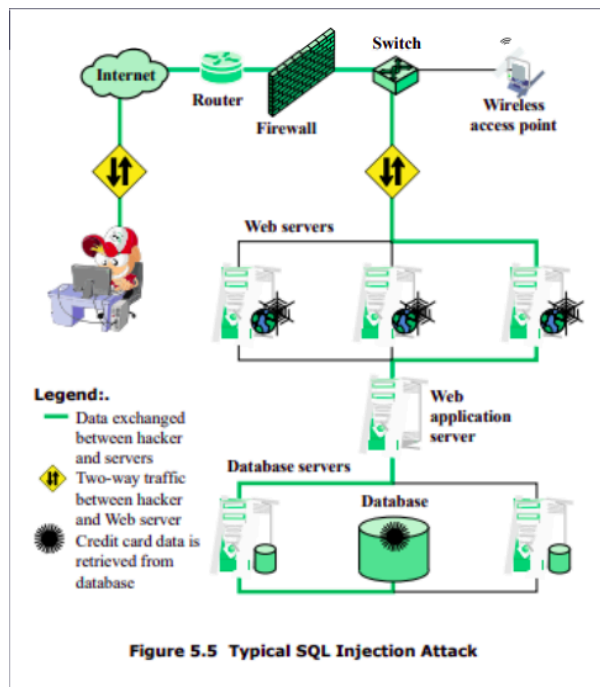
L'applicazione non restituisce direttamente i dati o gli errori. L'attaccante procede "alla cieca" ponendo al database domande con risposta Vero/Falso e deducendo le informazioni dal comportamento del sistema.

- **Boolean-based:** L'attaccante inietta condizioni logiche. Se la condizione è vera, la pagina si carica normalmente; se è falsa, si carica in modo diverso (es. scompare un elemento). Inviando query carattere per carattere, si può ricostruire l'intero database.
- **Time-Based:** Sfrutta le tempistiche di risposta dicendo al database di "dormire" (es. tramite **SLEEP()**) per un tot di secondi se una condizione è vera. Misurando il tempo di risposta del server, l'attaccante deduce il risultato.
- **Illegal/Incorrect Queries:** Generazione volontaria di errori logici per ricognizione preliminare senza estrazione diretta.

Out-of-Band SQLi

Si verifica quando l'attaccante non può usare lo stesso canale per l'attacco e per la risposta (es. server troppo lento o vincoli applicativi). Sfrutta configurazioni di rete permissive per forzare il database a inviare i dati rubati tramite un altro protocollo (es. richieste HTTP esterne o DNS verso un server controllato dall'hacker).

Anatomia di un Attacco: Flusso Logico



1. **Iniezione:** L'hacker inietta un comando SQL in un'applicazione web vulnerabile. Il comando si nasconde nel traffico HTTP legittimo accettato dal firewall.
2. **Inoltro:** Il Web Server riceve il traffico e inoltra il codice malevolo al Web Application Server.
3. **Esecuzione:** Il Web Application Server invia la query manipolata al Database Server.
4. **Estrazione:** Il Database Server esegue la query. Se l'attacco ha successo, il DB estrae i dati richiesti (es. tabella carte di credito) e li restituisce all'Application Server.
5. **Generazione Pagina:** L'Application Server popola dinamicamente una pagina web inserendo al suo interno i dati appena esfiltrati.
6. **Esfiltrazione:** Il Web Server invia la pagina renderizzata all'hacker, completando il furto di informazioni.

Difese e Contromisure

Per proteggere un'applicazione si utilizza un approccio di *Difesa in Profondità*.

Difesa Primaria

- **Prepared Statements (Query Parametrizzate):** È la difesa fondamentale. Invece di concatenare le stringhe, l'applicazione invia al database la struttura della query contenente dei segnaposto (placeholder). Il database pre-compila la query. Solo in un secondo momento vengono inviati i parametri.
 - **Risultato:** Qualsiasi dato inviato dall'utente viene interpretato dal database strettamente come **dato letterale** e mai come codice eseguibile. (Es. inserendo `admin' #`, il DB cercherà letteralmente un utente chiamato `admin' #`).

Difese Secondarie e Architetture

- **Validazione e Sanificazione dell'Input:** Controllare sempre che l'input rispetti il formato atteso. Si raccomanda un approccio **Allow-list** (accettare solo i caratteri o i formati esplicitamente consentiti) piuttosto che **Deny-list** (tentare di bloccare caratteri malevoli, tecnica facilmente bypassabile).
- **Principio del Minimo Privilegio (PoLP):** L'applicazione web non deve mai collegarsi al DBMS con i privilegi di root/amministratore. Deve utilizzare un account dedicato con i soli permessi DML necessari (es. `SELECT`, `INSERT`) sulle specifiche tabelle, revocando qualsiasi privilegio distruttivo DDL (come `DROP`).
- **Stored Procedures:** Spostano la logica SQL direttamente all'interno del motore del database. Se implementate correttamente (senza concatenazioni interne), fungono in modo simile ai Prepared Statements, isolando l'input utente dai comandi operativi.